LICEO RUSSELL · INFORMATICA · CLASSE SECONDA

Verifica di informatica

Introduzione a Python

0 1 0 1 1 0 0 1

DURATA	PUNTEGGIO	ESERCIZIO	SNIPPET
90 minuti	110 punti	80 punti	30 punti

Istruzioni generali

- La prova è divisa in due parti: **scrittura del codice** e **lettura del codice**.
- Per gli esercizi scrivi un programma in **Python**, in un file separato. Sono valutate la correttezza, la gestione dei casi limite e la chiarezza del codice.
- Nella parte di lettura spiega a parole che cosa fa ciascun frammento e riporta il suo output, senza riscrivere il codice.
- Il programma deve stampare **soltanto** il risultato richiesto e non deve chiedere dati con `input()` : usa le variabili già presenti nel template.
- Non è consentito consultare appunti, libri o Internet. È possibile usare il **cheatsheet** messo a disposizione.

Istruzioni per la consegna

- Aprite il **file explorer** (icona cartella sulla barra degli strumenti in basso) e navigate nella cartella `Questo Pc -> temi_xx` .
- Create una nuova cartella, denominata `cognome-nome` , **tutto minuscolo**. Se avete più nomi o più cognomi, metteteli tutti, concatenati dal carattere `-` (trattino). Ad esempio:

Vostro nome	Nome cartella
Marini Mattia	`marini-mattia`
Olaizola Elisa Maitea	`olaizola-elisa-maitea`

- Dentro a questa cartella, va creato **un file per ciascun esercizio**. Il nome del file è quello indicato nell'intestazione dell'esercizio, trasformato in modo che sia **tutto minuscolo e senza spazi**: l'estensione deve essere `.py` per gli esercizi di programmazione e `.txt` per le risposte alle domande e agli snippet.

Esercizio 1 **Esempio esercizio** **0 punti**

Ad esempio, per questo esercizio va creato un file `esercizio1.py`

Per questa prova nello specifico mi aspetto che consegniate i seguenti file:

Parte	File da consegnare
Scrittura del codice	`esercizio1.py` , `esercizio2.py` , `esercizio3.py` , `esercizio4.py`
Lettura del codice	`snippet1.txt` , `snippet2.txt` , `snippet3.txt`

Riepilogo dei punteggi

Voce	Dettaglio	File risposta	Punteggio
Scrittura del codice			
E1	Media degli elementi	`esercizio1.py`	**16**
E2	Tutti dentro l'intervallo	`esercizio2.py`	**23**
E3	L'elemento è nella lista?	`esercizio3.py`	**16**
E4	La lista è palindroma?	`esercizio4.py`	**25**
Lettura del codice			
S1	Snippet 1	`snippet1.txt`	**10**
S2	Snippet 2	`snippet2.txt`	**10**
S3	Snippet 3	`snippet3.txt`	**10**
		Totale	**110 punti**

1. Scrittura del codice

Per ciascun esercizio scrivi un file separato (`esercizio1.py` … `esercizio5.py`), eseguilo e verifica il risultato con gli esempi. Il programma deve stampare **soltanto** il risultato richiesto e non deve chiedere dati con `input()`: usa le variabili già presenti nel template.

Prima di consegnare

- Il programma stampa **solo** il risultato richiesto, senza messaggi in più?
- Hai provato i casi limite: lista vuota, un solo elemento, valori duplicati, numeri negativi?
- I nomi delle variabili sono chiari e il codice è rientrato in modo corretto?
- Ogni esercizio è nel file indicato nell'intestazione, con il nome corretto?

È **fondamentale** che il programma funzioni per qualsiasi input gli venga dato. Gli esercizi strutturati per funzionare solo su un input specifico verranno valutati **0 punti**. Ad esempio, se venisse chiesto di calcolare la media della lista `l`, il seguente programma varrebbe zero punti:

```
l = [4, 7, 10, 3]
print(6.0)
```

Funzioni e operatori non consentiti

- Sono vietati `sum()`, `max()`, `min()`, `sorted()`, `reversed()`, `all()`, `any()`, `list.reverse()`, `list.sort()`, `list.count()` e `list.index()`.
- Sono consentiti `len()`, `append()`, `pop()`, `remove()` e i normali cicli.
- Nell'esercizio **L'elemento è nella lista?** non usare nemmeno l'operatore `in`: la ricerca va fatta con un ciclo.

Esercizio 1 — Media degli elementi

★★☆☆☆ **16 punti**

Consegna questo esercizio nel file `esercizio1.py`

Consegna. Data la lista di numeri `l`, calcola e stampa la **media** dei suoi elementi. Se la lista è vuota, stampa `0`.

Nota. La media è la somma degli elementi divisa per il numero di elementi. Calcola la somma con un ciclo e usa `len(l)` per il conteggio: non usare `sum()`. Attenzione al caso limite della lista vuota.

TEMPLATE DI PARTENZA

```
l = [4, 7, 10, 3]

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = [4, 7, 10, 3]`	`6.0`	La somma è `24`, divisa per `4` elementi.
`l = [2, 3]`	`2.5`	La media può essere un numero decimale.
`l = []`	`0`	La media non è definita: stampa `0` per convenzione.

Esercizio 2 — Tutti dentro l'intervallo

★★★☆☆ 23 punti

Consegna questo esercizio nel file `esercizio2.py`

Consegna. Date la lista `l` e i due estremi `a` e `b`, stampa `True` se **tutti** gli elementi di `l` sono compresi nell'intervallo fra `a` e `b`, estremi inclusi, altrimenti stampa `False`. Se la lista è vuota, stampa `True`.

Nota. Gli estremi possono essere scritti in qualsiasi ordine: `a` può essere maggiore di `b` oppure minore. Prima di controllare, assicurati che `a` sia il limite inferiore e `b` quello superiore. Non usare `min()` né `max()`.

TEMPLATE DI PARTENZA

```
l = [4, 6, 2, 9]
a = 10
b = 1

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = [4, 6, 2, 9], a = 1, b = 10`	`True`	Tutti gli elementi stanno fra `1` e `10`.
`l = [4, 12, 2, 9], a = 10, b = 1`	`False`	Dopo lo scambio l'intervallo è `[1, 10]`: `12` è fuori.
`l = [], a = 7, b = 2`	`True`	Non ci sono elementi fuori dall'intervallo.

Esercizio 3 — L'elemento è nella lista?

★☆☆☆☆ 16 punti

Consegna questo esercizio nel file `esercizio3.py`

Consegna. Date la lista di interi `l` e il valore `a`, cerca `a` in `l`. Se lo trovi, stampa `Trovato all'indice N`, dove `N` è l'indice della **prima** occorrenza (ossia l'indice minore che contiene il valore `a`). Se non lo trovi, stampa `Non trovato`. Il valore `a` può essere un numero o una stringa.

Nota. Non usare `list.index()` né l'operatore `in`: la ricerca va fatta con un ciclo.

TEMPLATE DI PARTENZA

```
l = [5, 8, 2, 9, 8]
a = 8

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = [5, 8, 2, 9, 8], a = 8`	`Trovato all'indice 1`	Si considera la prima occorrenza.
`l = [5, 8, 2], a = 4`	`Non trovato`	`4` non compare nella lista.
`l = [], a = 1`	`Non trovato`	La lista è vuota.

Esercizio 4 — La lista è palindroma? ★★☆☆☆ 25 punti

Consegna questo esercizio nel file `esercizio4.py`

Consegna. Stabilisci se la lista `l` è **palindroma**: si legge allo stesso modo da sinistra a destra e da destra a sinistra. Stampa `True` oppure `False`. Una lista vuota e una lista con un solo elemento sono palindrome.

Nota. Consiglio il seguente approccio: confronta il primo elemento con l'ultimo, il secondo con il penultimo e così via. Basta fermarsi al centro della lista: se trovi due elementi diversi puoi concludere subito che la lista non è palindroma. Non usare `reversed()` né `list.reverse()`.

TEMPLATE DI PARTENZA

```
l = [1, 2, 3, 2, 1]

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = [1, 2, 3, 2, 1]`	`True`	La sequenza è simmetrica.
`l = [1, 2, 3]`	`False`	Gli estremi `1` e `3` sono diversi.
`l = []`	`True`	Non c'è nessuna coppia da confrontare.

2. Lettura del codice

Per ciascun frammento di codice spiega a parole che cosa fa e, se il programma stampa qualcosa, riporta anche l'output. Non devi riscrivere il codice: va dimostrato di averne compreso l'intento, descrivendone il funzionamento.

Snippet 1 — Snippet 1 — ★☆☆☆☆ 10 punti

Consegna questo esercizio nel file `snippet1.txt`

```
l = [3, 8, 0, -4, 7, 10]
i = len(l) - 1
while i >= 0:
    if l[i] % 2 == 0:
        print(l[i])
    i = i - 1
```

Snippet 2 — Snippet 2 — ★☆☆☆☆ 10 punti

Consegna questo esercizio nel file `snippet2.txt`

```
l = [3, -1, 0, 7]
b = False
for i in range(len(l)):
    if l[i] < 0:
        b = True
print(b)
```

Snippet 3 — Snippet 3 — ★☆☆☆☆ 10 punti

Consegna questo esercizio nel file `snippet3.txt`

Cosa succede ai valori di `a` e `b`?

```
a = 3
b = 8
t = a
a = b
b = t
print(a, b)
```